

Total Allocated Marks for Final Evaluation (40%)

Final Submission Deadline: 2026-06-06

Group Assignment | INTE 21323

Task Management System

Software Requirements Specification (SRS)

A Task Management System (TMS) is a software application designed to help individuals or teams plan, organize, track, and complete tasks efficiently. It's widely used in personal productivity, team collaboration, and project management. A Task Management System (TMS) is essential in project management because it provides a structured way to organize, assign, and track tasks throughout the project lifecycle.

In any project, multiple activities need to be coordinated among different team members, and without a centralized system, tasks can easily be overlooked, duplicated, or delayed. A TMS ensures that every task is clearly defined, assigned to a responsible individual, and given a deadline, which improves accountability and reduces confusion within the team.

It also enables real-time progress tracking, allowing managers to monitor what has been completed, what is in progress, and what is pending, thereby making it easier to identify bottlenecks early and take corrective action.

Additionally, by centralizing all project-related information such as task details, updates, and communication, a TMS enhances collaboration and keeps everyone aligned with project goals. Tools like Trello and Jira demonstrate how such systems can improve efficiency and productivity by streamlining workflows and ensuring that deadlines are consistently met, ultimately contributing to the successful completion of projects.

Popular examples include:

- Trello
- Asana
- Jira

Introduction

This document defines the functional and non-functional requirements for the Task Management System, a full-stack web application that allows users to create, manage, and track tasks collaboratively in real time.

The system enables:

- User Registration, Authentication and role-based access control
- Create tasks
- Assign tasks to people
- Set deadlines and priorities
- Track progress (e.g., To Do → In Progress → Done)
- Collaborate (comments, attachments, updates)
- Real-time Notifications updates using WebSockets
- Secure API communication
- Deployment in a cloud environment

Functional Requirements

User Authentication

The system shall allow registered users to log in using valid credentials (email and password). Credentials shall be validated against stored records with proper input validation and secure handling.

On success: access is granted

On failure: appropriate error message is returned

JWT-Based Session Management

Upon successful authentication:

- The system shall generate a JWT (JSON Web Token) containing user ID, role, and expiration time.
- The token shall be securely signed to prevent tampering.
- The token shall be securely stored on the client (e.g., HTTP-only cookies or secure storage).
- The client shall include the token in the Authorization header for all protected API requests.

The system shall:

- Validate token signature, expiration, and integrity on each request
- Support token expiration and optional refresh mechanism (gap filled)
- Reject invalid/expired tokens with appropriate responses

Authorization & Role-Based Access Control (RBAC)

Access to system features shall be restricted based on authentication status and user roles:

- Protected routes require a valid JWT
- Unauthorized access → 401 Unauthorized
- Forbidden actions → 403 Forbidden

Roles:

- **Administrator (Admin):** Full access to user management and system configuration. Can create, update, deactivate users, and assign roles.
- **Project Manager:** Can create/manage projects and tasks, assign tasks, set priorities, deadlines, and monitor progress.
- **Collaborator:** Can view assigned tasks, update task status, add comments/attachments. Cannot modify restricted fields or delete tasks.

User Management

Accessible only to Administrators. Other users do not have ability to update others or their profile.

Core Features

The system shall allow:

- Create, view, update, deactivate users
- Assign roles to users
- View users via searchable and filterable list

Validation & Constraints

- Required fields: name, email, role
- Email must be valid and unique
- All inputs must be validated before submission

User Onboarding

Upon user creation:

- System shall send an email with username and temporary password

On first login:

- Users must reset password (mandatory) before accessing the system
- Password policies must be enforced
 - Minimum length and complexity enforced

Storing Passwords

- Passwords shall be stored using secure hashing (e.g., bcrypt)
- Plain-text passwords shall never be stored

All operations shall include:

- Validation
- Error handling
- Success confirmations

Task Management

Core Features

Authorized users (primarily Project Managers) shall be able to:

- Create, view, update, assign, delete tasks

The system shall provide:

- Task views (table/board/Kanban)
- Filtering and sorting capabilities

Task Attributes

Each task shall include:

- Title (required)
- Description
- Assigned user(s)
- Due date
- Priority (Low, Medium, High)
- Status (To Do, In Progress, Completed)

Validation Rules

- Title is mandatory
- Due date must be valid (past dates restricted unless explicitly allowed)
- Priority must match predefined values
- Assignment limited to existing users

Permissions

Project Managers:

- full task control

Collaborators:

- Can view assigned tasks
- Can update status
- Can add comments/attachments
- Cannot delete or modify restricted fields

System Behavior

- All operations shall include validation and error handling
- UI shall provide clear feedback and confirmations
- Task updates shall reflect immediately (linked to real-time module)

Real-Time Features (WebSocket-Based)

The system shall use WebSockets to push updates without polling.

Events and Notifications

Users shall receive real-time notifications for:

- Task assignments
- Status changes
- Comments
- Approaching deadlines
- Administrative updates
- Client Behavior

WebSocket connection established after authentication. UI shall update dynamically (tasks, statuses, notifications)

Notification Handling

Notifications shall be:

Role-based and user-specific

Displayed via in-app alerts or notification panel

Offline users:

Notifications shall be stored and delivered upon reconnection

Reliability & Security

- Implement reconnection strategies (e.g., retry with backoff)
- Ensure secure transmission (e.g., WSS)
- Prevent unauthorized subscription to events

Validation & Error Handling

The system shall enforce validation at both:

- Frontend (immediate feedback)
- Backend (final validation layer)

Validation includes:

- Required fields
- Data formats (email, dates)
- Business rules
- Error Handling (Backend)

The API shall return Standard HTTP status codes with Structured error response as required.

Structured error response should have:

- Error code
- Message
- Detailed Description (if applicable)

Error codes:

- 400 – Bad Request
- 401 – Unauthorized
- 403 – Forbidden
- 500 – Internal Server Error

Frontend Handling

- Display user-friendly error messages
- Map input field-level errors to inputs
- Show global errors via alerts/notifications
- Maintain consistent error UI patterns

Security

- The system shall prevent SQL Injection attacks by ensuring that user inputs cannot manipulate database queries.
- All database interactions shall use parameterized queries or ORM frameworks (e.g., EF Core) instead of dynamic query concatenation
- Input data shall be validated and sanitized before processing
- Stored procedures (if used) shall avoid dynamic SQL execution
-
- Store passwords using bcrypt/scrypt hashing
- Use HTTPS for deployed services
- Sanitise and validate all inputs to avoid XSS/SQLi
- Use best practices with referring to owasp top 10 recommendations
- The system shall ensure all data transmitted between client and server is encrypted.
- All communication shall occur over HTTPS (TLS)

System Architecture

Uses MVC pattern.

Deployment Requirements

- Docker containerization, CI/CD, cloud hosting.
- Deploy both backend and frontend on a cloud platform (azure, aws, gcp or similar)
- Ensure CORS and environment configuration for production
- Provide a working demo link for both the frontend and the backend

Testing Requirements

Functional testing.

Git & Documentation Requirements

Version Control

- Every contributor must use feature branches
- Use pull requests and merge commits
- Follow meaningful commit messages

API Documentation

- Use Swagger/OpenAPI for documenting REST endpoints

README.md

- Project overview
- Technologies used
- Setup instructions
- API usage (link to Swagger UI)
- Team member contributions

Folder Structure

- Follow clean, modular folder structure (e.g., /controllers, /services, /routes)

Deliverables

- Source code, API documentations, ER/Class diagrams ,DB design, deployment diagrams.
- After initiation of projects each team must publish their source code to a GitHub public repository and share the link.
- make sure to demonstrate the website from a proper hosted url.

Grading Criteria

Total Marks: 100

Category	Marks (%)
Frontend	20
Backend	20
Database	10
Security	15
Real-Time Notifications	10
DevOps & Deployment	20
Documentation	5
Total Marks	100